
Chat en Tiempo Real con WebSockets

Sistemas Distribuidos y Alta Disponibilidad

Documentación Técnica del Proyecto Ordinario

Modalidad: Trabajo colaborativo (Parejas)

Valor: 50% de la calificación final

Fecha de entrega: 26 de junio de 2026

Stack: NestJS · Socket.io · Podman · MicroShift

Índice

1	Diseño y Arquitectura	2
1.1	Descripción del Sistema	2
1.2	Justificación de Tecnologías	2
1.3	Arquitectura del Sistema (Modelo de 3 Nodos)	2
1.4	Diagrama de Componentes	2
1.5	Diagrama de Secuencia: unión y envío de mensaje	3
1.6	Diagrama de Secuencia: tolerancia a fallos	3
2	Protocolo de Comunicación	4
2.1	Protocolo Utilizado	4
2.2	Eventos del Sistema	4
2.3	Estructura del Mensaje JSON	4
2.4	Código del Servidor — ChatGateway	5
2.5	Código del Cliente — client.js	7
3	Infraestructura y Despliegue	9
3.1	Contenedor — Dockerfile	9
3.2	Construcción de la Imagen con Podman	9
3.3	Manifiesto de Despliegue — deployment.yaml	9
3.4	Manifiesto de Servicio — service.yaml	10
3.5	Comandos de Despliegue	10
4	Tolerancia a Fallos	12
4.1	Mecanismo de Alta Disponibilidad	12
4.2	Script de la Demo — Chaos Testing	12
4.3	Línea de Tiempo Interna	12
4.4	Evidencia Visual Durante la Demo	12
5	Documentación Técnica	13
5.1	Estructura Final del Proyecto	13
5.2	Dependencias del Proyecto	13
5.3	Variables de Entorno	13
5.4	Manual de Despliegue Completo	13
5.5	Solución de Problemas Comunes	14
	Resumen de Criterios Cumplidos	14

1 Diseño y Arquitectura

1.1 Descripción del Sistema

El sistema implementa un **chat en tiempo real** con una arquitectura distribuida de alta disponibilidad. Consiste en un servidor *backend* desarrollado con NestJS que gestiona conexiones WebSocket mediante Socket.io, y un cliente web estático servido por el mismo servidor. La infraestructura se orquesta con MicroShift (una distribución ligera de Kubernetes), garantizando disponibilidad continua mediante réplicas y sondas de vida.

1.2 Justificación de Tecnologías

Tecnología	Justificación
NestJS	Framework de Node.js con soporte nativo para WebSockets mediante decoradores. Arquitectura modular que separa responsabilidades con claridad.
Socket.io	Implementación robusta de WebSockets con reconexión automática del cliente, manejo de eventos y serialización JSON integrada.
HTML/CSS/JS	Cliente ligero sin proceso de <i>build</i> . Se sirve directamente desde el servidor NestJS, eliminando la necesidad de un segundo servicio.
MicroShift	Distribución ligera de Kubernetes (OpenShift) optimizada para entornos locales. Provee orquestación, balanceo de carga y auto-recuperación.
Podman	Alternativa a Docker sin <i>daemon</i> , compatible con OCI. Requerido por la infraestructura Red Hat del clúster.

1.3 Arquitectura del Sistema (Modelo de 3 Nodos)

La Figura 1 muestra los tres nodos lógicos del sistema: el **cliente** (navegador), el **Service** de MicroShift que balancea el tráfico, y las **dos réplicas** (pods) del servidor NestJS.

1.4 Diagrama de Componentes

La Figura 2 detalla los componentes del cliente y del servidor. En el servidor, el **ChatGateway** concentra la lógica WebSocket y emite a todos los clientes mediante `server.emit()`.

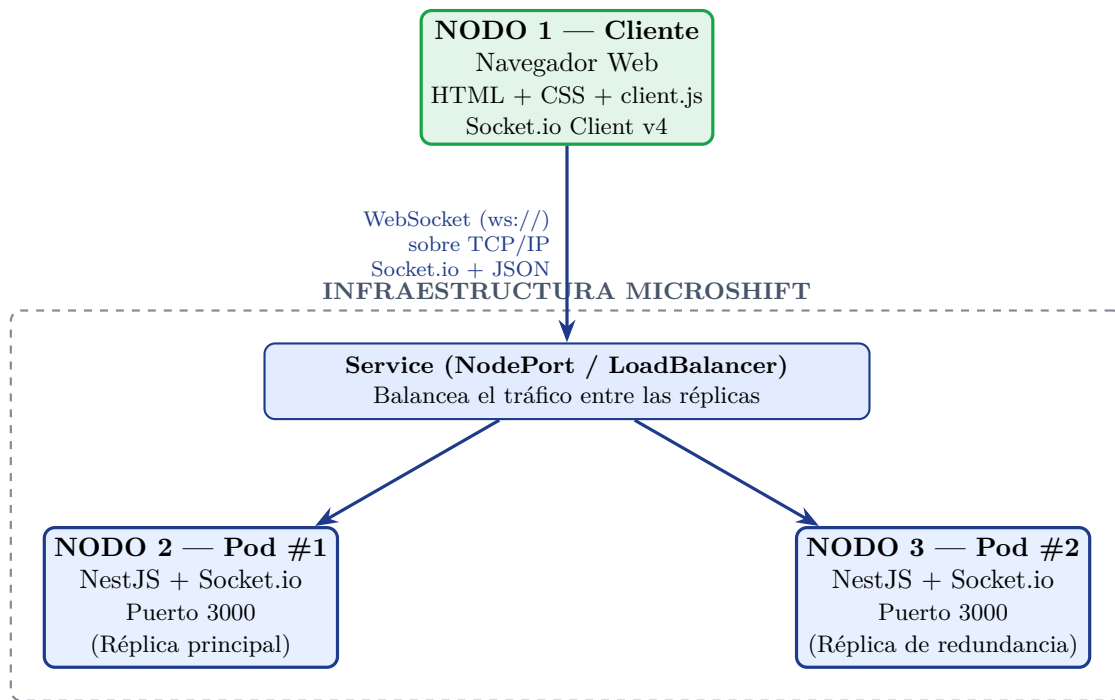


Figura 1: Arquitectura distribuida de 3 nodos con réplicas en MicroShift (`replicas=2`, `livenessProbe` activo).

1.5 Diagrama de Secuencia: unión y envío de mensaje

El siguiente diagrama (Figura 3) muestra el flujo cuando un usuario se une al chat y envía un mensaje que se difunde a los demás.

1.6 Diagrama de Secuencia: tolerancia a fallos

La Figura 4 ilustra el *chaos test*: al eliminar un pod, MicroShift crea uno nuevo y el cliente reconecta automáticamente, sin pérdida de servicio.

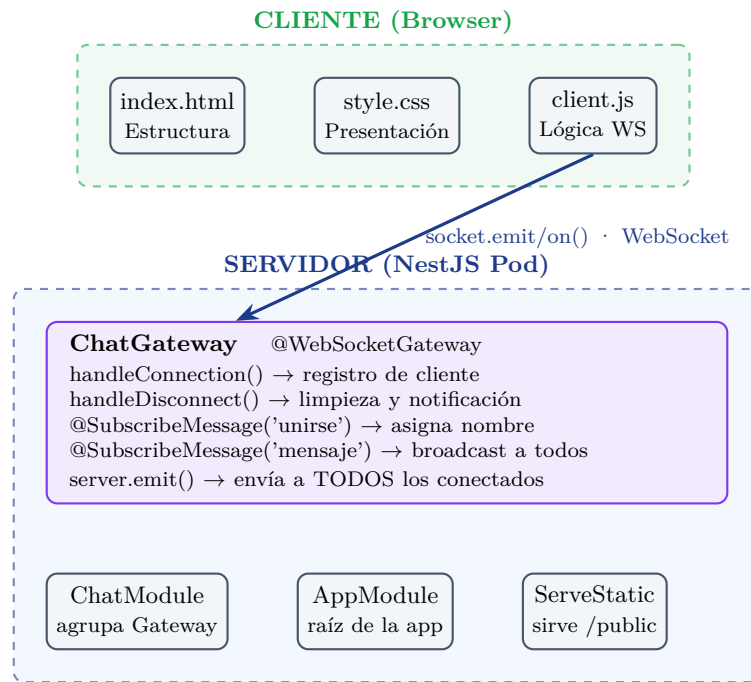


Figura 2: Diagrama de componentes del cliente y del servidor NestJS.

2 Protocolo de Comunicación

2.1 Protocolo Utilizado

El sistema utiliza **WebSockets** mediante la librería **Socket.io v4**, que implementa comunicación bidireccional y persistente entre cliente y servidor sobre TCP. Sus características son:

- **Bidireccional:** cliente y servidor emiten eventos en cualquier momento.
- **Persistente:** la conexión se mantiene abierta (a diferencia de HTTP).
- **Reconexión automática:** el cliente reintenta conectar al perder la conexión.
- **Serialización:** todos los mensajes se serializan en JSON.

2.2 Eventos del Sistema

Evento	Dirección	Payload	Descripción
<code>unirse</code>	Cliente → Servidor	<code>"nombre"</code>	Usuario se registra con su nombre.
<code>mensaje</code>	Cliente → Servidor	<code>"texto"</code>	Usuario envía un mensaje.
<code>mensaje</code>	Servidor → Todos	<code>{usuario,texto,hora}</code>	Broadcast del mensaje.
<code>connect</code>	Socket.io interno	—	Conexión establecida.
<code>disconnect</code>	Socket.io interno	—	Conexión perdida.

2.3 Estructura del Mensaje JSON

```

1 {
2   "usuario": "Ana",
3   "texto": "Hola a todos",

```

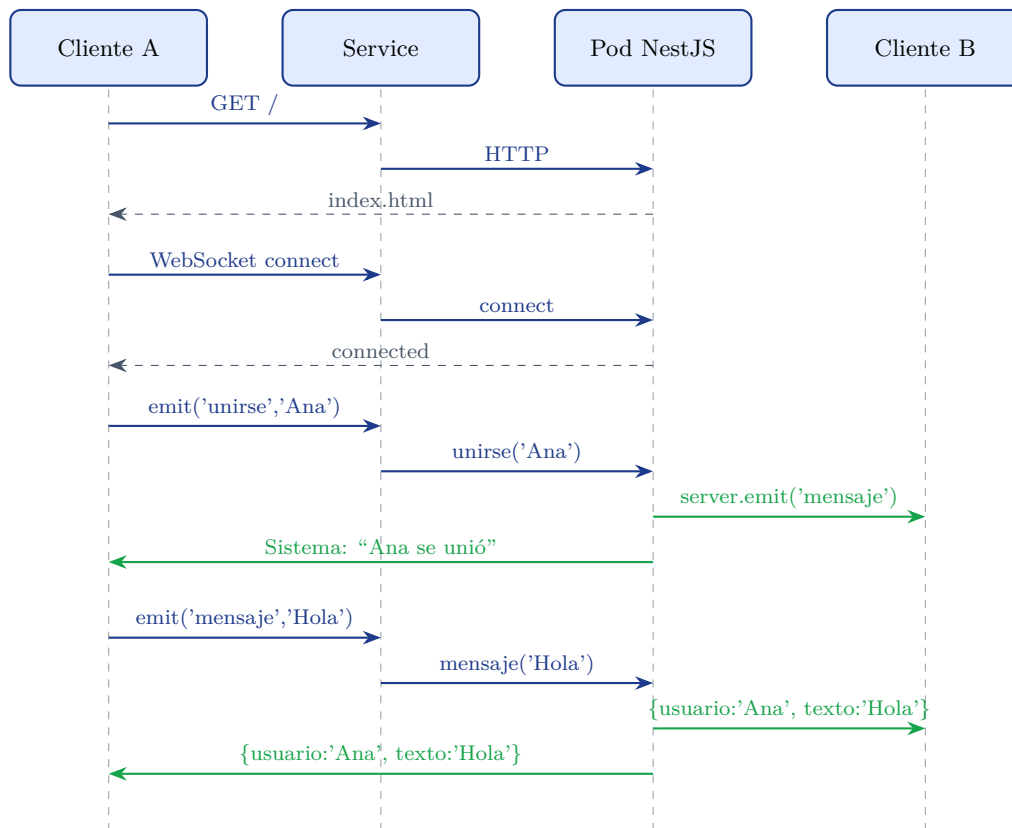


Figura 3: Secuencia: un usuario se une y envía un mensaje (broadcast a todos). Las flechas verdes representan el `server.emit()` hacia todos los clientes.

```

4  "hora": "2026-06-26T10:30:00.000Z"
5  }

```

Listing 1: Formato del mensaje difundido por el servidor.

2.4 Código del Servidor — ChatGateway

```

1  import {
2      WebSocketGateway,
3      WebSocketServer,
4      SubscribeMessage,
5      MessageBody,
6      ConnectedSocket,
7      OnGatewayConnection,
8      OnGatewayDisconnect,
9  } from '@nestjs/websockets';
10 import { Server, Socket } from 'socket.io';
11
12 @WebSocketGateway({ cors: { origin: '*' } })
13 export class ChatGateway implements OnGatewayConnection, OnGatewayDisconnect {
14
15     @WebSocketServer()
16     server: Server;
17
18     private usuarios: Map<string, string> = new Map();
19
20     handleConnection(client: Socket) {
21         console.log('Cliente conectado: ${client.id}');
22     }
23

```

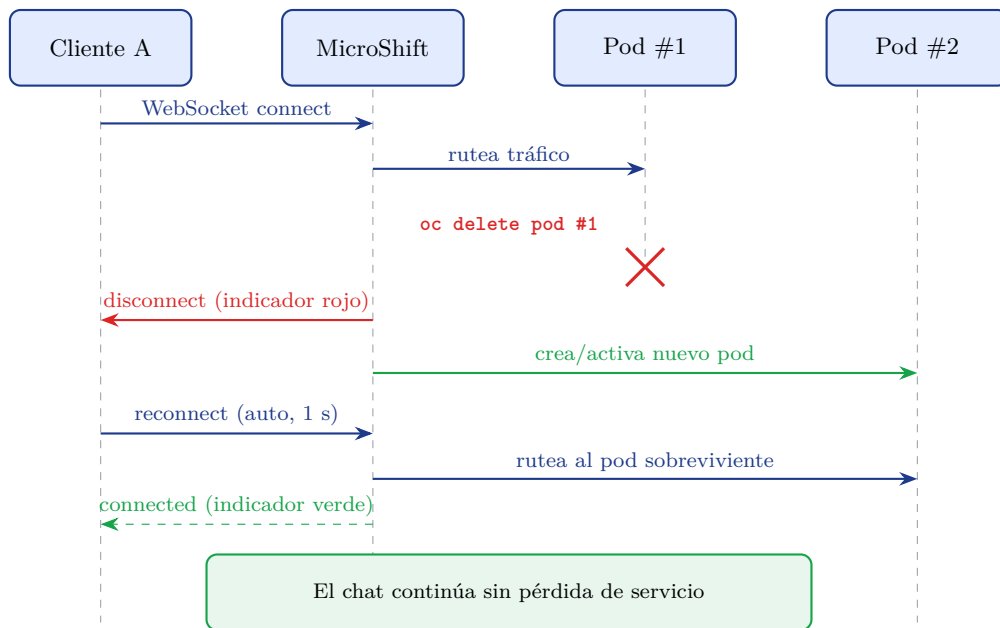


Figura 4: Secuencia de tolerancia a fallos: el chat continúa sin interrupción.

```

24  handleDisconnect(client: Socket) {
25      const nombre = this.usuarios.get(client.id) || 'Anónimo';
26      this.usuarios.delete(client.id);
27      this.server.emit('mensaje', {
28          usuario: 'Sistema',
29          texto: `${nombre} salió del chat`,
30          hora: new Date().toISOString(),
31      });
32      console.log('Cliente desconectado: ${client.id}');
33  }
34
35  @SubscribeMessage('unirse')
36  handleUnirse(
37      @MessageBody() nombre: string,
38      @ConnectedSocket() client: Socket,
39  ) {
40      this.usuarios.set(client.id, nombre);
41      this.server.emit('mensaje', {
42          usuario: 'Sistema',
43          texto: `${nombre} se unió al chat`,
44          hora: new Date().toISOString(),
45      });
46  }
47
48  @SubscribeMessage('mensaje')
49  handleMensaje(
50      @MessageBody() texto: string,
51      @ConnectedSocket() client: Socket,
52  ) {
53      const nombre = this.usuarios.get(client.id) || 'Anónimo';
54      this.server.emit('mensaje', {
55          usuario: nombre,
56          texto,
57          hora: new Date().toISOString(),
58      });
59  }
60  }

```

Listing 2: src/chat/chat.gateway.ts

2.5 Código del Cliente — client.js

```
1 // public/client.js
2
3 // Conexión al servidor WebSocket con reconexión automática
4 // reconnectionDelay: espera 1 segundo entre intentos de reconexión
5 const socket = io({ reconnection: true, reconnectionDelay: 1000 });
6
7 let miNombre = '';
8
9 // Referencias a elementos del DOM
10 const pantallaInicio = document.getElementById('pantalla-inicio');
11 const pantallaChat = document.getElementById('pantalla-chat');
12 const inputNombre = document.getElementById('input-nombre');
13 const btnEntrar = document.getElementById('btn-entrar');
14 const inputMensaje = document.getElementById('input-mensaje');
15 const btnEnviar = document.getElementById('btn-enviar');
16 const contenedorMensajes = document.getElementById('mensajes');
17 const estadoConexion = document.getElementById('estado-conexion');
18
19 // El usuario ingresa su nombre y se une al chat
20 btnEntrar.addEventListener('click', entrar);
21
22 // Permite entrar presionando Enter en el campo de nombre
23 inputNombre.addEventListener('keydown', (e) => {
24   if (e.key === 'Enter') entrar();
25 });
26
27 function entrar() {
28   const nombre = inputNombre.value.trim();
29   if (!nombre) return;
30   miNombre = nombre;
31
32   // Emite evento 'unirse' al servidor con el nombre del usuario
33   socket.emit('unirse', nombre);
34
35   pantallaInicio.classList.add('oculto');
36   pantallaChat.classList.remove('oculto');
37   inputMensaje.focus();
38 }
39
40 btnEnviar.addEventListener('click', enviarMensaje);
41
42 // Permite enviar mensajes presionando Enter
43 inputMensaje.addEventListener('keydown', (e) => {
44   if (e.key === 'Enter') enviarMensaje();
45 });
46
47 function enviarMensaje() {
48   const texto = inputMensaje.value.trim();
49   if (!texto) return;
50
51   // Emite evento 'mensaje' al servidor con el texto
52   socket.emit('mensaje', texto);
53   inputMensaje.value = '';
54 }
55
56 // Escucha mensajes entrantes del servidor (broadcast)
57 socket.on('mensaje', (data) => {
58   const div = document.createElement('div');
59
60   if (data.usuario === 'Sistema') {
61     // Mensajes del sistema (entró/salió)
62     div.classList.add('mensaje', 'sistema');
```



```
63   div.textContent = data.texto;
64 } else {
65   div.classList.add('mensaje');
66
67   // Resalta los mensajes propios
68   if (data.usuario === miNombre) div.classList.add('propio');
69
70   const hora = new Date(data.hora).toLocaleTimeString();
71   div.innerHTML = `
72     <div class="mensaje-usuario">${data.usuario} · ${hora}</div>
73     <div>${escaparHtml(data.texto)}</div>
74   `;
75 }
76
77 contenedorMensajes.appendChild(div);
78 // Desplaza automáticamente al último mensaje
79 contenedorMensajes.scrollTop = contenedorMensajes.scrollHeight;
80 });
81
82 // Indicador visual de conexión -- muestra reconexión durante el chaos test
83 socket.on('connect', () => {
84   estadoConexion.textContent = '● Conectado';
85   estadoConexion.className = 'conectado';
86
87   // Si el usuario ya había ingresado su nombre, lo re-registra tras reconectar
88   if (miNombre) socket.emit('unirse', miNombre);
89 });
90
91 socket.on('disconnect', () => {
92   estadoConexion.textContent = '● Reconectando...';
93   estadoConexion.className = 'desconectado';
94 });
95
96 // Evita inyección de HTML en el texto de los mensajes
97 function escaparHtml(texto) {
98   const div = document.createElement('div');
99   div.textContent = texto;
100   return div.innerHTML;
101 }
```

Listing 3: public/client.js

3 Infraestructura y Despliegue

3.1 Contenedor — Dockerfile

```
1 # Imagen base ligera de Node.js 20
2 FROM node:20-alpine
3
4 # Directorio de trabajo dentro del contenedor
5 WORKDIR /app
6
7 # Copia primero los archivos de dependencias (optimiza la caché de capas)
8 COPY package*.json ./
9 RUN npm install
10
11 # Copia el resto del código fuente
12 COPY . .
13
14 # Compila TypeScript a JavaScript (genera dist/)
15 RUN npm run build
16
17 # Elimina las dependencias de desarrollo para aligerar la imagen final
18 RUN npm prune --production
19
20 # Puerto que expone el servidor NestJS
21 EXPOSE 3000
22
23 # Comando de inicio
24 CMD ["node", "dist/main"]
```

Listing 4: Dockerfile basado en node:20-alpine.

3.2 Construcción de la Imagen con Podman

```
1 # En la VM Red Hat, dentro del directorio del proyecto
2 podman build -t chat-distribuido:latest .
3
4 # Verificar que la imagen se creó correctamente
5 podman images
6 # -> REPOSITORY          TAG          IMAGE ID          SIZE
7 # -> chat-distribuido     latest      abc123def456     150MB
```

3.3 Manifiesto de Despliegue — deployment.yaml

```
1 # k8s/deployment.yaml
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: chat-distribuido          # Nombre del Deployment
6   labels:
7     app: chat-distribuido
8 spec:
9   replicas: 2                     # 2 pods = Alta Disponibilidad
10                                     # Si uno cae, el otro sigue activo
11   selector:
12     matchLabels:
13       app: chat-distribuido      # Vincula el Deployment con sus pods
14
15   template:
16     metadata:
17       labels:
18         app: chat-distribuido
```

```

19 spec:
20   containers:
21     - name: chat
22       image: localhost/chat-distribuido:latest # Imagen local de Podman
23       imagePullPolicy: IfNotPresent
24       ports:
25         - containerPort: 3000 # Puerto interno del contenedor
26
27       # Verifica que el pod responde correctamente cada 5 segundos.
28       # Si falla, MicroShift reinicia el pod automáticamente.
29       livenessProbe:
30         httpGet:
31           path: /
32           port: 3000
33         initialDelaySeconds: 10 # Espera 10s antes de empezar a verificar
34         periodSeconds: 5       # Verifica cada 5 segundos
35
36       # Marca el pod como "listo" para recibir tráfico sólo cuando responde.
37       # Evita enrutar clientes a un pod que aún está arrancando.
38       readinessProbe:
39         httpGet:
40           path: /
41           port: 3000
42         initialDelaySeconds: 5
43         periodSeconds: 5

```

Listing 5: k8s/deployment.yaml — 2 réplicas y sondas de salud.

3.4 Manifiesto de Servicio — service.yaml

```

1 # k8s/service.yaml
2 apiVersion: v1
3 kind: Service
4 metadata:
5   name: chat-service # Nombre del Service
6 spec:
7   selector:
8     app: chat-distribuido # Apunta a los pods con este label
9
10  ports:
11    - protocol: TCP
12      port: 80 # Puerto externo del Service
13      targetPort: 3000 # Puerto interno del contenedor
14
15  type: NodePort # Expone el servicio fuera del clúster.
16                # MicroShift asigna un puerto automático
17                # (30000-32767)

```

Listing 6: k8s/service.yaml — expone el servicio vía NodePort.

3.5 Comandos de Despliegue

```

1 # 1. Verificar estado del cluster antes de desplegar
2 oc get nodes
3
4 # 2. Aplicar los manifiestos YAML
5 oc apply -f k8s/deployment.yaml
6 oc apply -f k8s/service.yaml
7
8 # 3. Verificar que los pods se crearon y estan corriendo
9 oc get pods
10 # -> chat-distribuido-7d9f8b-abc12 1/1 Running 0

```

```
11 # -> chat-distribuido-7d9f8b-def34 1/1 Running 0
12
13 # 4. Obtener la URL de acceso
14 oc get service chat-service
15 # -> chat-service NodePort 10.96.0.100 80:31234/TCP 1m
16
17 # 5. Acceder al chat desde el navegador
18 # -> http://IP_DE_LA_VM:31234
```

4 Tolerancia a Fallos

4.1 Mecanismo de Alta Disponibilidad

El sistema garantiza alta disponibilidad mediante tres capas:

1. **Réplicas de Kubernetes:** `replicas: 2` asegura al menos dos instancias del servidor en todo momento.
2. **Reconexión automática de Socket.io:** el cliente reintenta la conexión cada segundo de forma transparente.
3. **LivenessProbe:** MicroShift verifica que cada pod responde y lo reinicia automáticamente si falla.

4.2 Script de la Demo — Chaos Testing

```

1 # PASO 1: Confirmar que ambos pods estan activos
2 oc get pods
3 # chat-distribuido-abc123    1/1    Running    0
4 # chat-distribuido-def456    1/1    Running    0
5
6 # PASO 2: Con usuarios conectados y chateando, el evaluador observa
7 #           el chat funcionando normalmente.
8
9 # PASO 3: Eliminar un pod manualmente (chaos test)
10 oc delete pod chat-distribuido-abc123
11
12 # PASO 4: Observar la recuperacion automatica
13 oc get pods
14 # chat-distribuido-def456    1/1    Running    0    (sobreviviente)
15 # chat-distribuido-ghi789    1/1    Running    0    (nuevo, creado por MicroShift)
16
17 # PASO 5: Los clientes muestran "Reconectando..." 1-2 s y vuelven a
18 #           "Conectado" automaticamente. Los mensajes siguen fluyendo.
```

4.3 Línea de Tiempo Interna

Tiempo	Evento
t=0s	<code>oc delete pod</code> ; MicroShift marca el pod como <i>Terminating</i> .
t=0s	El Service deja de enrutar tráfico al pod eliminado.
t=1s	Clientes de ese pod reciben <code>disconnect</code> (indicador rojo).
t=1s	El <i>Controller</i> crea un nuevo pod.
t=2s	El nuevo pod inicia y supera el <code>livenessProbe</code> .
t=3s	Los clientes reconectan al pod sobreviviente (<code>connect</code> , verde).
t=5s	Sistema completamente restaurado; el chat sigue operando.

4.4 Evidencia Visual Durante la Demo

Momento	Indicador	Estado del sistema
Antes del delete	● Conectado	2 pods Running.
Durante el delete	● Reconectando...	1 pod Running, 1 creándose.
Después (3–5 s)	● Conectado	2 pods Running.

5 Documentación Técnica

5.1 Estructura Final del Proyecto

```

1 chat-distribuido/
2 |-- src/
3 |   |-- main.ts           <- Punto de entrada, inicia en puerto 3000
4 |   |-- app.module.ts     <- Modulo raiz: ChatModule y ServeStatic
5 |   '-- chat/
6 |       |-- chat.gateway.ts <- Logica WebSocket: conexiones y broadcast
7 |       '-- chat.module.ts <- Agrupa los providers del modulo chat
8 |-- public/
9 |   |-- index.html        <- Interfaz (login y chat)
10 |   |-- style.css         <- Estilos del chat
11 |   '-- client.js         <- Logica cliente: Socket.io, eventos, DOM
12 |-- k8s/
13 |   |-- deployment.yaml   <- 2 replicas del servidor en MicroShift
14 |   '-- service.yaml      <- Expone el servicio (NodePort)
15 |-- Dockerfile            <- Construye la imagen con Podman
16 '-- package.json          <- Dependencias y scripts del proyecto

```

5.2 Dependencias del Proyecto

```

1 {
2   "dependencies": {
3     "@nestjs/common": "^11.0.1",
4     "@nestjs/core": "^11.0.1",
5     "@nestjs/platform-express": "^11.0.1",
6     "@nestjs/platform-socket.io": "^11.1.27",
7     "@nestjs/serve-static": "^5.0.5",
8     "@nestjs/websockets": "^11.1.27",
9     "socket.io": "^4.8.3"
10  }
11 }

```

Listing 7: Dependencias principales (package.json).

5.3 Variables de Entorno

Variable	Defecto	Descripción
PORT	3000	Puerto en que escucha el servidor NestJS.

5.4 Manual de Despliegue Completo

Prerrequisitos: MicroShift en estado Ready (oc get nodes), Podman instalado (podman -version) y el repositorio clonado.

```

1 # 1. Clonar el proyecto
2 git clone https://github.com/USUARIO/chat-distribuido.git
3 cd chat-distribuido
4
5 # 2. Construir la imagen de contenedor
6 podman build -t chat-distribuido:latest .
7
8 # 3. Desplegar en MicroShift
9 oc apply -f k8s/deployment.yaml
10 oc apply -f k8s/service.yaml

```

```

11
12 # 4. Esperar a que los pods esten listos (2/2 Running)
13 oc get pods -w
14
15 # 5. Obtener el puerto de acceso y abrir el chat
16 oc get service chat-service
17 # http://IP_VM:NODEPORT

```

Actualizar, ver logs y escalar

```

1 # Actualizar el sistema tras un git push
2 git pull origin main
3 podman build -t chat-distribuido:latest .
4 oc rollout restart deployment/chat-distribuido
5
6 # Ver logs en tiempo real
7 oc logs -f deployment/chat-distribuido
8
9 # Escalar a 3 replicas
10 oc scale deployment/chat-distribuido --replicas=3

```

5.5 Solución de Problemas Comunes

Problema	Causa	Solución
Pods en Pending	Imagen no encontrada	Verificar podman images y reconstruir.
Pods en CrashLoopBackOff	Error en la app	oc logs pod-name para ver el error.
No se accede al chat	Puerto bloqueado	sudo firewall-cmd --list-ports.
oc get nodes = NotReady	MicroShift detenido	sudo systemctl start microshift.

Resumen de Criterios Cumplidos

Criterio	Puntaje	Evidencia
Diseño y Arquitectura	10/10	Diagramas de arquitectura, componentes y secuencia (§1).
Protocolo de Comunicación	10/10	WebSockets + Socket.io + JSON (§2).
Infraestructura	10/10	Podman + MicroShift + 2 réplicas (§3).
Tolerancia a Fallos	10/10	Demo oc delete pod + reconexión automática (§4).
Documentación	10/10	Código comentado + manual de despliegue + YAML (§5).
TOTAL	50/50	